

Course Outline: MCP Client — The Other Side of MCP

Title: MCP Client — The Other Side of MCP **Skill:** __ — Skill: Proveer de herramientas a un Agente mediante CLIs **Learnpack:** + MCP Client **File:** s__-w20d59-mcp-client **Prerequisite:** MCP Server (FastMCP, tool definitions, running a server with stdio) **Target Audience:** Beginner developers who just built a Python MCP server and now want to build a TypeScript client that connects to it **Total Lessons:** 8 **Format:** Mixed (12.5% hands-on)

Course Description

You built the MCP server — now you build the client. This course implements a TypeScript MCP client using the official MCP TypeScript SDK. Students connect to the Python server from the previous learnpack, discover its tools, and call the `inspect_file` tool. The course covers: connecting to a server via stdio transport, listing available tools, calling a tool with arguments, and handling responses. The hands-on challenge produces a working TypeScript client that interacts with the Python server from the previous learnpack.

Course Philosophy

This course emphasizes:

- The parallel structure of client and server — you already understand the server side
 - TypeScript as the implementation language — highlighting that MCP is language-agnostic
 - Concrete output: a real tool call that returns real data from the server you built
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - MCP Client Fundamentals	2
02 - Building an MCP Client	3
03 - Assessment	1
04 - Outro	1
Total	8

00.0 The Other Side of MCP 

Learning Objectives:

- Explain how an MCP client completes the client-server pair
- Connect MCP client concepts to the architecture and server from previous learnpacks

Content Outline:

- Recap: you built a Python MCP server that exposes `inspect_file`. Now you need something that talks to it.
- The client's role: connect to a server, discover its tools, call them, and use the results
- Why TypeScript: the MCP ecosystem has excellent TypeScript tooling, and TypeScript clients can connect to servers written in any language (Python, Go, Rust, etc.)
- The MCP TypeScript SDK: Anthropic's official TypeScript library for building both clients and servers
- Course overview: client fundamentals, connecting to a server, listing tools, calling tools, handling results

Transitions: This lesson sets the stage. Section 01 explains what MCP clients do and what real-world clients look like.

Key Principles:

- A client doesn't care what language the server is written in — it speaks the MCP protocol
- The client is the bridge between the AI model's intent and the server's capability

Examples: The flow you'll build:

1. Start your Python MCP server: `python my_server.py`
2. Run your TypeScript client: `npm ts-node client.ts`
3. Client connects to server (via stdio)
4. Client requests: "What tools do you have?"
5. Server responds: `["inspect_file"]`
6. Client calls: `inspect_file({path: "package.json"})`
7. Server returns: `{path: "package.json", exists: true, size_bytes: 843}`

01.0 MCP Client Fundamentals

Learning Objectives:

- Describe the responsibilities of an MCP client: connect, discover, call, handle responses
- Explain the client's connection lifecycle: connect → initialize → use → disconnect
- Preview what lesson 01.1 covers (use cases and real-world examples)

Content Outline:

- **What an MCP client does:**
 - Establishes a connection to an MCP server (via transport — stdio or SSE)
 - Sends the initialization handshake and receives server capabilities
 - Requests the list of available tools (`tools/list`)
 - Calls specific tools with arguments (`tools/call`)
 - Receives results and handles errors

- Disconnects when done
- **Connection lifecycle:**
 1. Create a client instance
 2. Connect via transport (for stdio: spawn the server as a subprocess)
 3. Client automatically handles the MCP initialization handshake
 4. Use the client: list tools, call tools
 5. Close the connection
- **What the TypeScript SDK handles for you:** JSON-RPC 2.0 message serialization, protocol handshake, error parsing, reconnection logic
- Preview: 01.1 covers where clients are used and examples

Transitions: You know what the client does. Lesson 01.1 shows where clients are used.

Key Principles:

- The client must connect BEFORE it can do anything — the transport creates the link to the server
- stdio transport: the client spawns the server as a child process; communication is over pipes

Examples: A minimal client interaction:

```
1. Client spawns: python my_server.py (or uses a running process)
2. MCP handshake completes automatically
3. client.listTools() → [{name: "inspect_file", description: "Inspect a file..."}]
4. client.callTool("inspect_file", {path: "package.json"}) → {path: ..., exists: true, size_bytes: ...}
5. client.close()
```

01.1 MCP Client Use Cases

Learning Objectives:

- Give three examples of applications that embed MCP clients
- Explain why MCP clients are built into AI tools like Claude Desktop and Cursor
- Describe the difference between using an existing MCP client vs building your own

Content Outline:

- **Real-world MCP clients:**
 - **Claude Desktop:** embeds an MCP client that connects to locally configured servers, giving Claude access to your filesystem, databases, and custom tools
 - **Cursor:** embeds an MCP client to give the AI code editor access to project-specific tools
 - **Continue.dev:** open-source coding assistant that uses MCP clients to access tools
 - **Custom applications:** building your own client lets you integrate any MCP server into your AI app
- **When to use existing clients vs building your own:**

- **Existing clients (Claude Desktop, Cursor):** use when you just want to give Claude/AI access to your server in the context of an existing app
- **Building your own:** use when you're building a custom AI application and need programmatic control over tool calls
- **In this course:** you'll build your own client — this gives you the deepest understanding of how the protocol works
- **The connection between previous and this learnpack:** your Python server from the previous learnpack is a fully compliant MCP server; this TypeScript client will connect to it directly

Transitions: You know where clients are used. Section 02 builds one.

Key Principles:

- Building a client is the best way to understand MCP end-to-end
- A custom client gives you full control: you decide when to list tools, what to call, and how to handle results

Examples: How Claude Desktop uses an MCP client internally:

1. Claude Desktop reads config: "connect to filesystem server at /path/to/server.py"
2. Claude Desktop's MCP client spawns the server
3. Claude gets a list of available tools
4. When Claude needs to read a file: "read_file('/readme.txt')"
5. Claude Desktop's client calls the tool, gets the result, adds it to Claude's context

02.0 Building an MCP Client 📖

Learning Objectives:

- Install the TypeScript MCP SDK and set up a basic project
- Describe the structure of a TypeScript MCP client: imports, client creation, transport, connection, usage, close
- Preview what lessons 02.1 (discovering and calling tools) and 02.2 (hands-on) cover

Content Outline:

- **Installation:**

```
npm install @modelcontextprotocol/sdk
npm install -D typescript ts-node @types/node
```

- **Basic client structure:**

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from
"@modelcontextprotocol/sdk/client/stdio.js";
```

```

async function main() {
  // Create the transport (spawns the server as a subprocess)
  const transport = new StdioClientTransport({
    command: "python",
    args: ["my_server.py"]
  });

  // Create the client
  const client = new Client({
    name: "my-client",
    version: "1.0.0"
  });

  // Connect (handshake happens automatically)
  await client.connect(transport);

  // Use the client here (list tools, call tools)

  // Clean up
  await client.close();
}

main().catch(console.error);

```

- **What `StdioClientTransport` does:** spawns the server process and creates stdio pipes for communication
- **What `client.connect()` does:** sends the initialization handshake and waits for the server to acknowledge
- Overview of 02.1 (listTools, callTool) and 02.2 (hands-on)

Transitions: You know the structure. Lesson 02.1 goes deep into listing tools and making calls.

Key Principles:

- The transport creates the physical connection; the client provides the protocol interface
- `await client.connect()` must be called before any other operations

Examples: The TypeScript SDK handles a lot:

- Transport management (starting/stopping the server process)
- Protocol handshake (initialize → initialized)
- Request/response matching (tracking request IDs)
- Error serialization/deserialization

02.1 Discovering and Calling Tools

Learning Objectives:

- Use `client.listTools()` to get available tools with their schemas
- Use `client.callTool()` to invoke a specific tool with arguments

- Parse and use the tool's response correctly

Content Outline:

- **Listing tools:** `client.listTools()`

```
const response = await client.listTools();
const tools = response.tools;
// tools is an array of tool objects with: name, description,
// inputSchema

for (const tool of tools) {
  console.log(`Tool: ${tool.name}`);
  console.log(`Description: ${tool.description}`);
  console.log(`Schema: ${JSON.stringify(tool.inputSchema)}`);
}
```

- **Calling a tool:** `client.callTool()`

```
const result = await client.callTool({
  name: "inspect_file",
  arguments: {
    path: "package.json"
  }
});

// result.content is an array of content items
// For text tools, it's: [{type: "text", text: "..."}]
const content = result.content[0];
if (content.type === "text") {
  const data = JSON.parse(content.text);
  console.log("File size:", data.size_bytes);
}
```

- **Response structure:**
 - `result.content`: array of content items (usually one)
 - `content.type`: "text" for JSON responses from FastMCP tools
 - `content.text`: the JSON string returned by the tool
- **Error handling:**
 - If `result.isError === true`: the tool raised an exception; check `content.text` for the error message
 - Use try/catch around `callTool` for network/protocol errors

Transitions: With listing and calling understood, lesson 02.2 builds the full client that connects to your Python server.

Key Principles:

- Always check `result.isError` before parsing the response
- FastMCP tools return JSON strings in `content.text` — parse with `JSON.parse()`

Anti-patterns:

- **Assuming `result.content[0].text` is always valid JSON:** it might be an error message if `result.isError` is true
- **Not closing the client:** always `await client.close()` at the end — this stops the server subprocess too

Examples: A complete list + call cycle:

```
// 1. List tools
const { tools } = await client.listTools();
console.log("Available tools:", tools.map(t => t.name));

// 2. Call a specific tool
const result = await client.callTool({
  name: "inspect_file",
  arguments: { path: "README.md" }
});

// 3. Handle result
if (result.isError) {
  console.error("Tool error:", result.content[0].text);
} else {
  const data = JSON.parse(result.content[0].text);
  console.log("File info:", data);
}
```

02.2 Your First MCP Client

Challenge Prompt: Build a TypeScript MCP client that connects to the Python MCP server from the previous learnpack, lists its available tools, and calls `inspect_file` with `path: "package.json"`. Print the tool list and the file inspection result to the console.

Solution Code:

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from
"@modelcontextprotocol/sdk/client/stdio.js";

async function main() {
  const transport = new StdioClientTransport({
    command: "python",
    args: ["my_server.py"]
  });
}
```

```

const client = new Client({
  name: "my-mcp-client",
  version: "1.0.0"
});

await client.connect(transport);

// List available tools
const { tools } = await client.listTools();
console.log("Available tools:");
for (const tool of tools) {
  console.log(` - ${tool.name}: ${tool.description}`);
}

// Call inspect_file
const result = await client.callTool({
  name: "inspect_file",
  arguments: { path: "package.json" }
});

if (result.isError) {
  console.error("Error:", result.content[0].text);
} else {
  const data = JSON.parse(result.content[0].text);
  console.log("File inspection result:", data);
}

await client.close();
}

main().catch(console.error);

```

Challenge Code:

```

import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from
"@modelcontextprotocol/sdk/client/stdio.js";

async function main() {
  // Step 1: create the transport (command: "python", args: ["my_server.py"])
  const transport = new StdioClientTransport({
    // your code here
  });

  const client = new Client({
    name: "my-mcp-client",
    version: "1.0.0"
  });

  // Step 2: connect to the server
  // your code here

```

```

// Step 3: list tools and print each tool name and description
const { tools } = // your code here
console.log("Available tools:");
for (const tool of tools) {
    console.log(` - ${tool.name}: ${tool.description}`);
}

// Step 4: call inspect_file with path: "package.json" and print the result
const result = // your code here

if (result.isError) {
    console.error("Error:", result.content[0].text);
} else {
    const data = JSON.parse(result.content[0].text);
    console.log("File inspection result:", data);
}

await client.close();
}

main().catch(console.error);

```

03.0 MCP Client Assessment 🧠

Learning Objectives:

- Demonstrate understanding of MCP client implementation
- Correctly identify the sequence of client operations
- Apply error handling patterns for tool responses

Question Format: Multiple choice

Questions:

1. What does `StdioClientTransport` do when created?
 - a) It connects directly to a running server via HTTP
 - b) It prepares to spawn the server as a subprocess with communication over stdin/stdout
 - c) It validates the server's MCP capabilities
 - d) It creates a WebSocket connection to the server
2. What method must be called before `client.listTools()` or `client.callTool()`?
 - a) `client.initialize()`
 - b) `client.authenticate()`
 - c) `await client.connect(transport)`
 - d) `client.handshake()`
3. What does `client.listTools()` return?

- a) The names of all tools as a plain string
 - b) An object with a `tools` array, each containing `name`, `description`, and `inputSchema`
 - c) A boolean indicating if any tools are available
 - d) The tool count as a number
4. When a FastMCP tool returns successfully, where is the JSON data in the response?
- a) `result.data`
 - b) `result.output.json`
 - c) `result.content[0].text` (as a JSON string to parse)
 - d) `result.json`
5. A tool call returns `result.isError === true`. What should you do?
- a) Retry the call immediately
 - b) Check `result.content[0].text` for the error message and handle accordingly
 - c) Restart the server
 - d) The client SDK automatically resolves the error
6. Why is `await client.close()` important?
- a) It saves the session state to disk
 - b) It sends a final message to the server and stops the server subprocess
 - c) It closes only the local client connection, leaving the server running
 - d) It is optional — the connection closes automatically
7. What TypeScript SDK package provides the `Client` class?
- a) `@anthropic/mcp-sdk`
 - b) `mcp-typescript`
 - c) `@modelcontextprotocol/sdk`
 - d) `@openai/mcp-client`

Evaluation Criteria:

- 7/7: Excellent — complete understanding of MCP client implementation
- 5–6/7: Good — minor gaps in response parsing or lifecycle
- 3–4/7: Needs review — revisit Sections 01 and 02
- Below 3/7: Restart recommended

Score Interpretation: Questions map to: StdioClientTransport (Q1), connect before use (Q2), listTools response (Q3), response parsing (Q4), error handling (Q5), close importance (Q6), SDK package (Q7).

04.0 The Full MCP Picture

Content Outline:

- Course recap: what students accomplished
 - Understood the client's role in MCP: connect, discover, call, handle

- Traced the client lifecycle: transport → connect → listTools → callTool → close
- Set up the TypeScript MCP SDK and created a working client structure
- Implemented tool discovery and calling, including error handling
- Built a TypeScript client that connects to a Python server, discovers its tools, and calls `inspect_file`
- Connection to bigger picture: you now understand MCP from both sides — server AND client. This is the complete picture. Any AI agent that supports MCP can now use your server; any MCP server that exists (Anthropic's filesystem server, GitHub server, database servers) you can now connect to with your client.
- The MCP ecosystem: hundreds of community MCP servers are already available (github.com/modelcontextprotocol/servers) — you can connect your client to any of them
- Next steps: Claude Code, Cursor, Continue.dev, and other AI tools all use MCP. You understand the protocol they're built on. Consider building your own MCP server that exposes something useful to you and connecting it to Claude Desktop.
- Resources for further exploration:
 - TypeScript MCP SDK: npmjs.com/package/@modelcontextprotocol/sdk
 - MCP community servers: github.com/modelcontextprotocol/servers
 - Claude Desktop MCP configuration: docs.anthropic.com/claude/docs/mcp
 - MCP specification: spec.modelcontextprotocol.io
- Encouragement: you've completed a full MCP implementation — server in Python, client in TypeScript. That's a real engineering achievement. Most developers don't know what MCP is; you've implemented both sides of it.

Note: This is conclusion only — no theory, exercises, or assessment.
